

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import shutil

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from google.colab import files

uploaded = files.upload()

uploaded_file = list(uploaded.keys())[0]

print("Uploaded file:", uploaded_file)
df = pd.read_excel(uploaded_file)

display(df.head())

print("\nDataset Shape:", df.shape)

print("\nColumn Names:")
print(df.columns.tolist())
!pip install xlrd
df = pd.read_excel(uploaded_file)
print("Dataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

# Remove completely empty columns
df = df.dropna(axis=1, how="all")

# Remove duplicate rows
df = df.drop_duplicates()

# Remove rows with missing values
df = df.dropna()

print("\nDataset After Cleaning:")
display(df.head())

print("\nFinal Dataset Shape:")
print(df.shape)
X = df[["Square_Footage"]]

```

```

Y = df["House_Price"]

print("X:")
display(X.head())

print("\nY:")
display(Y.head())
plt.figure(figsize=(8, 6))

plt.scatter(
    X["Square_Footage"],
    Y
)

plt.xlabel("Square Footage")
plt.ylabel("House Price")

plt.title("House Price vs Square Footage")

plt.grid(True)

plt.show()
X_train, X_test, Y_train, Y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)

print("Y_train:", Y_train.shape)
print("Y_test:", Y_test.shape)
SLR_model = LinearRegression()

SLR_model.fit(
    X_train,
    Y_train
)

print("Model trained successfully!")
slope = SLR_model.coef_[0]

intercept = SLR_model.intercept_

print("Slope:", slope)

print("Intercept:", intercept)

print("\nRegression Equation:")

```

```

print(
    f"House Price = {slope:.4f} × Square Footage + {intercept:.4f}"
)
Y_pred = SLR_model.predict(X_test)

print("Predicted Values:")

print(Y_pred)
results = pd.DataFrame({
    "Actual House Price": Y_test.values,
    "Predicted House Price": Y_pred
})

display(results.head(10))
MAE = mean_absolute_error(
    Y_test,
    Y_pred
)

MSE = mean_squared_error(
    Y_test,
    Y_pred
)

RMSE = np.sqrt(MSE)

R2 = r2_score(
    Y_test,
    Y_pred
)

print("Mean Absolute Error (MAE):", MAE)

print("Mean Squared Error (MSE):", MSE)

print("Root Mean Squared Error (RMSE):", RMSE)

print("R² Score:", R2)
n = len(Y_test)

p = 1

Adjusted_R2 = 1 - (
    (1 - R2) * (n - 1)
    /
    (n - p - 1)
)

print("Adjusted R² Score:", Adjusted_R2)
plt.figure(figsize=(9, 6))

```

```

# Scatter plot
plt.scatter(
    X["Square_Footage"],
    Y,
    label="Actual Data"
)

# Generate values for regression line
X_line = np.linspace(
    X["Square_Footage"].min(),
    X["Square_Footage"].max(),
    100
).reshape(-1, 1)

Y_line = SLR_model.predict(X_line)

# Plot regression line
plt.plot(
    X_line,
    Y_line,
    linewidth=2,
    label="Regression Line"
)

plt.xlabel("Square Footage")
plt.ylabel("House Price")

plt.title(
    "Simple Linear Regression - House Price Prediction"
)

plt.legend()

plt.grid(True)

plt.show()
plt.figure(figsize=(9, 6))

plt.scatter(
    X_test,
    Y_test,
    label="Actual Test Data"
)

# Sort test data for a clean line
X_test_sorted = np.sort(
    X_test.values,
    axis=0
)

```

```

)
Y_pred_line = SLR_model.predict(
    X_test_sorted
)

plt.plot(
    X_test_sorted,
    Y_pred_line,
    linewidth=2,
    label="Regression Line"
)

plt.xlabel("Square Footage")
plt.ylabel("House Price")
plt.title(
    "Test Data vs Regression Line"
)

plt.legend()
plt.grid(True)

plt.show()
new_square_footage = 2500

new_house = pd.DataFrame({
    "Square_Footage": [new_square_footage]
})

predicted_price = SLR_model.predict(
    new_house
)

print(
    "Square Footage:",
    new_square_footage
)

print(
    "Predicted House Price:",
    predicted_price[0]
)

<IPython.core.display.HTML object>

Saving house_pricing_expl.linear regression.et to
house_pricing_expl.linear regression (1).et
Uploaded file: house_pricing_expl.linear regression (1).et

```

```

{"summary":{"\n  \"name\": \"\")\n\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"Unnamed: 0\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n        \"samples\": [],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Unnamed: 1\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n        \"samples\": [],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Unnamed: 2\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n        \"samples\": [],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Unnamed: 3\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n        \"samples\": [],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Unnamed: 4\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n        \"samples\": [],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Unnamed: 5\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n        \"samples\": [],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Unnamed: 6\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n        \"samples\": [],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Unnamed: 7\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n        \"samples\": [],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Unnamed: 8\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n        \"samples\": [],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Square_Footage\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1767,\n        \"min\": 966,\n        \"max\": 4926,\n        \"num_unique_values\": 5,\n        \"samples\": [],\n        \"semantic_type\": \"\"

```

```
\n    \"description\\\": \\\"\\\"\\n        }\\n    },\\n    {\\n        \"column\\\":  
\\\"House_Price\\\",\\n        \"properties\\\": {\\n            \"dtype\\\":  
\\\"number\\\",\\n            \"std\\\": 390094.8796410908,\\n            \"min\\\":  
229698.9186636115,\\n            \"max\\\": 1041740.8589249004,\\n  
\\\"num_unique_values\\\": 5,\\n            \"samples\\\": [],\\n  
\\\"semantic_type\\\": \\\"\\\"\\\",\\n            \"description\\\": \\\"\\\"\\\"\\n        }\\  
n    }\\n ]\\n}\", \"type\": \"dataframe\"}
```

Dataset Shape: (1000, 11)

Column Names:

```
['Unnamed: 0', 'Unnamed: 1', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4',  
'Unnamed: 5', 'Unnamed: 6', 'Unnamed: 7', 'Unnamed: 8',  
'Square_Footage', 'House_Price']
```

```
Requirement already satisfied: xlrd in /usr/local/lib/python3.13/dist-packages (2.0.2)
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	0 non-null	float64
1	Unnamed: 1	0 non-null	float64
2	Unnamed: 2	0 non-null	float64
3	Unnamed: 3	0 non-null	float64
4	Unnamed: 4	0 non-null	float64
5	Unnamed: 5	0 non-null	float64
6	Unnamed: 6	0 non-null	float64
7	Unnamed: 7	0 non-null	float64
8	Unnamed: 8	0 non-null	float64
9	Square_Footage	1000 non-null	int64
10	House_Price	1000 non-null	float64

```
dtypes: float64(10), int64(1)
```

```
memory usage: 86.1 KB
```

None

Missing Values:

Unnamed: 0	1000
Unnamed: 1	1000
Unnamed: 2	1000
Unnamed: 3	1000
Unnamed: 4	1000
Unnamed: 5	1000
Unnamed: 6	1000
Unnamed: 7	1000
Unnamed: 8	1000
Square_Footage	0
House_Price	0

dtype: int64

Dataset After Cleaning:

```
{"summary":{"name": "Square_Footage", "rows": 5, "fields": [{"column": "Square_Footage", "properties": {"dtype": "number", "std": 1767, "min": 966, "max": 4926, "num_unique_values": 5, "samples": [4272, 4926, 3592], "semantic_type": "", "description": ""}], {"column": "House_Price", "properties": {"dtype": "number", "std": 390094.8796410908, "min": 229698.9186636115, "max": 1041740.8589249004, "num_unique_values": 5, "samples": [985260.854490162, 1041740.8589249004, 777977.3901185812], "semantic_type": "", "description": ""}], "type": "dataframe"}
```

Final Dataset Shape:

(1000, 2)

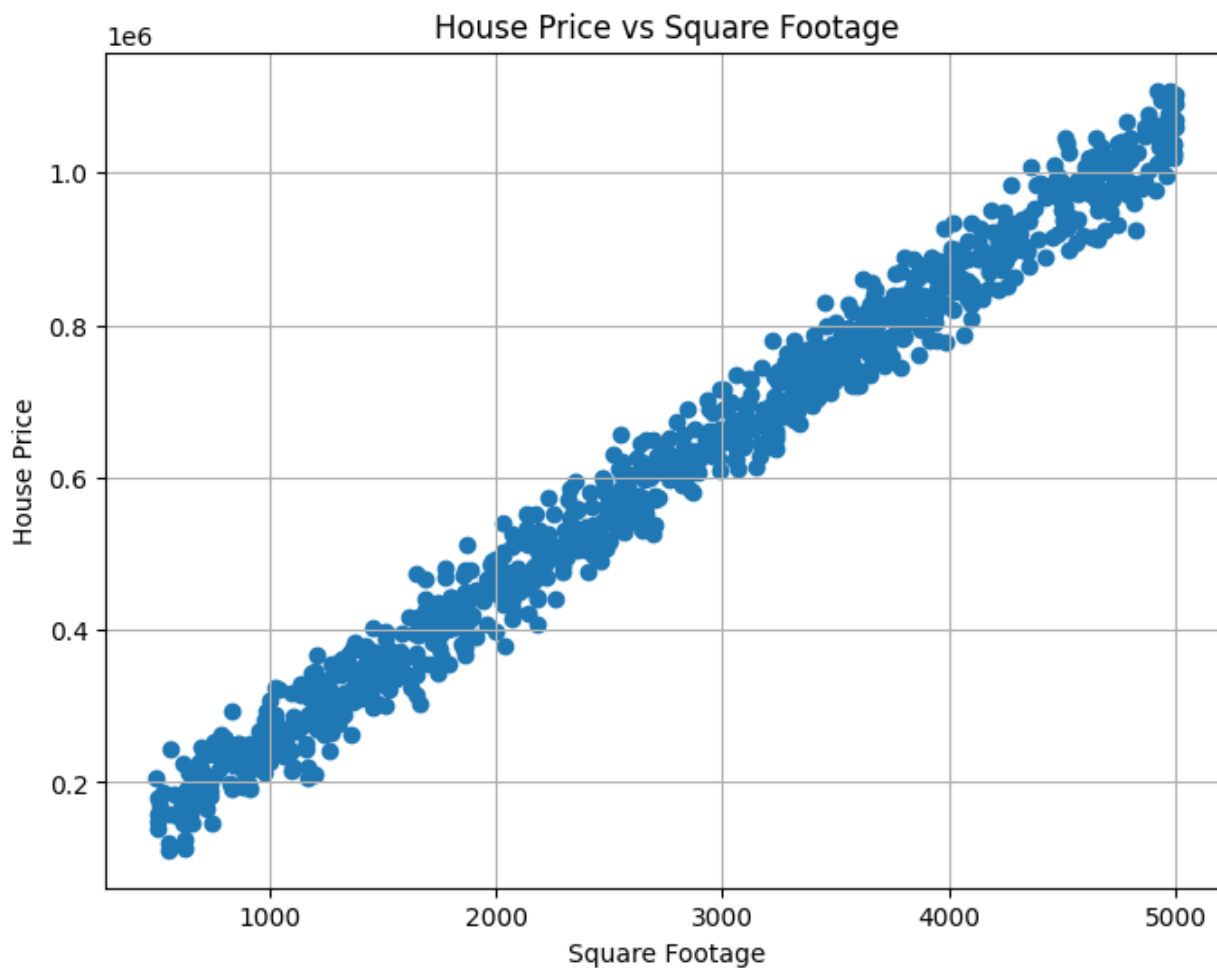
X:

```
{"summary":{"name": "Square_Footage", "rows": 5, "fields": [{"column": "Square_Footage", "properties": {"dtype": "number", "std": 1767, "min": 966, "max": 4926, "num_unique_values": 5, "samples": [4272, 4926, 3592], "semantic_type": "", "description": ""}], "type": "dataframe"}
```

Y:

```
0    2.623829e+05
1    9.852609e+05
2    7.779774e+05
3    2.296989e+05
4    1.041741e+06
```

Name: House_Price, dtype: float64



```
X_train: (800, 1)
X_test: (200, 1)
Y_train: (800,)
Y_test: (200,)
Model trained successfully!
Slope: 200.55687687000562
Intercept: 54229.058595876675
```

Regression Equation:
House Price = 200.5569 × Square Footage + 54229.0586

Predicted Values:

858863.24859834	517515.44416559	998450.83489986	1043375.57531874
785459.43166392	773426.01905172	984411.85351896	881927.28943839
812534.61004137	905191.88715531	778239.3840966	669537.55683305
210663.42255448	1041971.67718065	568657.44776744	231721.89462583
492646.39143371	396178.53365924	722484.57232674	330195.321169
747353.62505862	514306.53413567	719275.66229682	883932.85820709
444312.18410804	345838.75756486	452133.90230597	437092.13654072
238139.71468567	451532.23167536	270830.48561548	909203.02469271
762395.39082387	333805.34495266	481615.76320586	777838.27034286

476000.1706535	571866.35779736	249571.45666726	186797.15420695
320969.70483298	255187.04921962	498462.54086294	309738.51972826
1056010.65856155	929258.71237971	798696.18553734	811932.93941076
991030.23045567	734317.42806207	256791.50423458	728300.72175597
790272.7967088	700824.42962477	516713.21665811	796690.61676864
181983.78916207	308735.73534391	729905.17677093	238540.82843941
979799.04535095	359276.06831516	262607.65366381	485025.23011265
580089.18974903	587309.23731635	410418.07191701	743141.93064435
955331.10637281	711855.05785263	491041.93641875	860868.81736704
308936.29222078	565247.98086065	646874.62974674	717270.09352812
506083.702184	202841.70435655	588713.13545444	903787.98901722
570663.01653614	358273.28393081	534562.77869954	222095.16453607
155309.72453836	339822.05125876	1001459.18805291	859866.03298269
490039.1520344	178373.76537841	905793.55778592	800701.75430604
406406.93437961	748356.40944297	735721.32620016	1055408.98793094
747754.73881236	885537.31322205	824768.57953044	623008.36139921
903587.43214035	857659.90733712	211666.20693883	621203.34950738
499264.76837042	988021.87730262	523131.03671795	220089.59576737
276446.07816784	1005470.32559031	822361.897008	379131.19912529
875308.91250168	562640.74146134	777036.04283538	947108.27442114
252379.25294344	895565.15706555	226106.30207347	746150.2837974
194819.42928175	492846.94831058	840612.57280317	276044.9644141
364089.43336004	358072.72705394	520122.6835649	871899.44559489
931865.95177902	593727.05737619	718874.54854308	675353.70626228
615788.31383189	708245.03406897	696612.7352105	848634.84787797
551810.67011036	667331.43118748	702829.99839347	253181.48045092
450930.56104475	780244.9528653	367699.4571437	757983.13953273
857860.46421399	348245.4400873	960345.02829456	346841.54194921
675353.70626228	985214.08102644	549403.98758792	807320.13124275
1005269.76871344	909804.69532332	1014695.94192633	663721.40740382
269627.14435426	687988.78950509	1016300.39694129	762194.833947
201638.36309533	891955.13328189	779643.28223469	674551.4787548
1047186.15597927	981403.50036591	926049.80234979	201036.69246472
654094.67731406	808523.47250397	651687.99479162	256791.50423458
499866.43900103	466373.44056374	517515.44416559	506885.92969148
668935.88620244	1012088.70252702	698417.74710233	536768.90434511
1010484.24751206	813737.95130259	334005.90182953	434484.89714141
578885.84848781	454340.02795154	403799.6949803	569660.23215179
587910.90794696	335209.24309075	535766.11996076	526941.61737848
997247.49363864	974785.1234292	205248.38687899	809726.81376519
177772.0947478	954929.99261907	426863.73582035	695008.28019554]

```
{
  "summary": {
    "name": "Actual House Price",
    "rows": 10,
    "fields": [
      {
        "column": "Actual House Price",
        "dtype": "number",
        "std": 160626.79725457306,
        "min": 494537.5115454976,
        "max": 1040389.0481357794,
        "num_unique_values": 10,
        "samples": [
          792681.5184200833,
          494537.5115454976,
          724033.5604571347
        ],
        "semantic_type": "description"
      }
    ]
  }
}
```

```

n    },\n    {\n        \"column\": \"Predicted House Price\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 150276.43050984017, \n            \"min\": 517515.4441655897, \n            \"max\": 1043375.5753187444, \n            \"num_unique_values\": 10, \n            \"samples\": [\n                812534.6100413679, \n                517515.4441655897, \n                773426.0190517169\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }\n    ],\n    \"type\": \"dataframe\"}

```

Mean Absolute Error (MAE): 27179.533059260702

Mean Squared Error (MSE): 1081485546.488955

Root Mean Squared Error (RMSE): 32885.94755346051

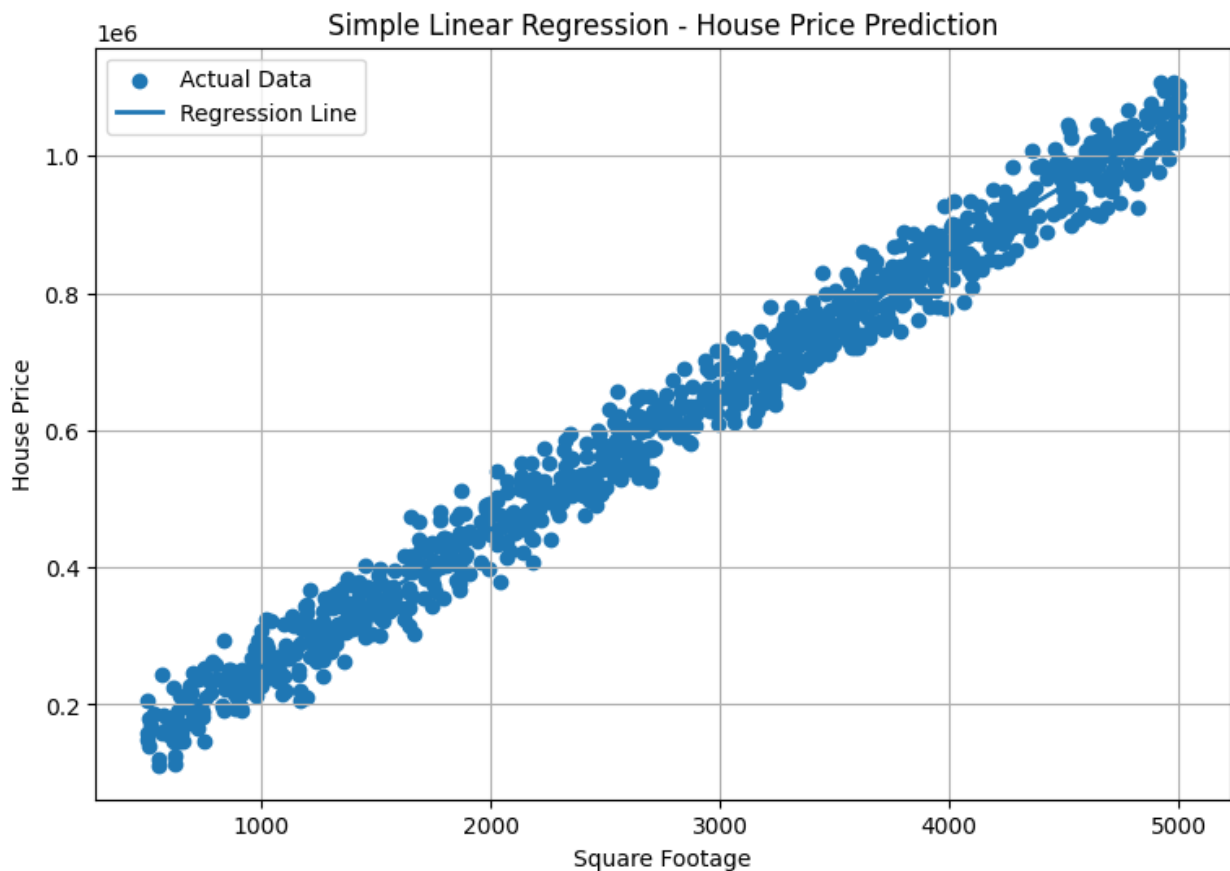
R² Score: 0.9832220800154357

Adjusted R² Score: 0.9831373430458167

```

/usr/local/lib/python3.13/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but LinearRegression was fitted with feature names
  warnings.warn(

```

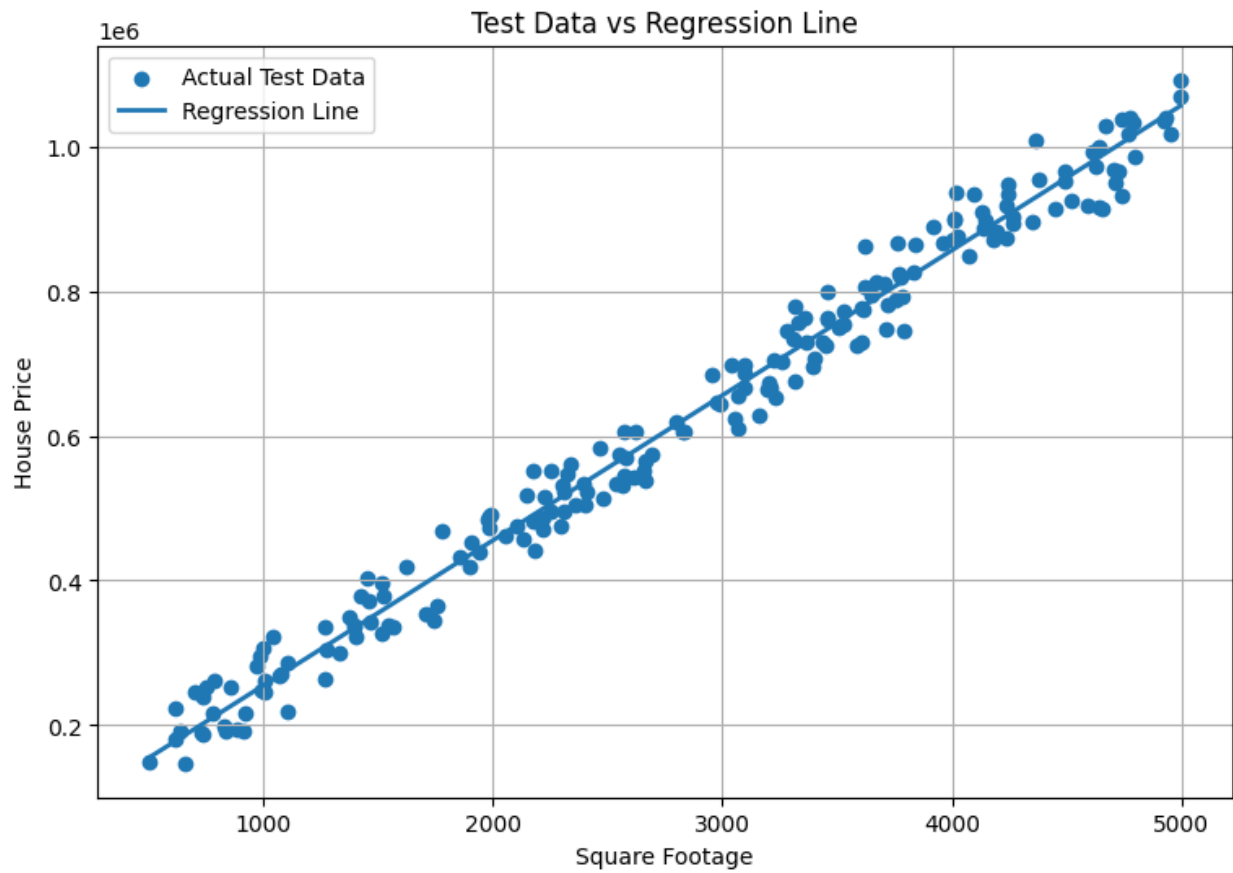


```

/usr/local/lib/python3.13/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,

```

```
but LinearRegression was fitted with feature names
warnings.warn()
```



Square Footage: 2500
Predicted House Price: 555621.2507708907

```

# House Price Prediction with Multiple Linear Regression

# This Google Colab experiment predicts house prices using Multiple Linear Regression (MLR).

# Pipeline: Import data -> Basic preprocessing -> Skip feature management -> Scatter plot -> Train/test split -> Train MLR -> Coefficients and intercept -> Accuracy metrics -> Visualization -> Prediction.

# > Variable names `x_MLR` and `y_MLR` are used for features and target to avoid conflicts with previous code.
# ## 1. Upload and import the dataset
import io
import pandas as pd
from google.colab import files

uploaded = files.upload()
file_name = next(iter(uploaded))

df = pd.read_csv(io.BytesIO(uploaded[file_name]))
print("Dataset shape:", df.shape)
display(df.head())
df.info()
# ## 2. Basic data preprocessing (feature management skipped)
# Basic preprocessing only
print("Missing values before preprocessing:")
print(df.isnull().sum())

# Remove duplicate rows
df = df.drop_duplicates()

# Fill numeric missing values with the median
numeric_cols = df.select_dtypes(include="number").columns
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())

# Fill categorical missing values with the mode
categorical_cols = df.select_dtypes(exclude="number").columns
for col in categorical_cols:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])

print("\nMissing values after preprocessing:")
print(df.isnull().sum())

# Feature management is intentionally skipped.
# We only prepare usable predictors for Multiple Linear Regression.
# ## 3. Define `x_MLR` and `y_MLR`

```

```

# Automatically identify the target column
target_col = "price" if "price" in df.columns else
df.select_dtypes(include="number").columns[-1]

# Use all remaining columns as predictors
x_MLR = df.drop(columns=[target_col])
y_MLR = df[target_col]

# Convert categorical predictors to dummy variables so sklearn can
train the model
x_MLR = pd.get_dummies(x_MLR, drop_first=True, dtype=float)

print("Target column:", target_col)
print("x_MLR shape:", x_MLR.shape)
print("y_MLR shape:", y_MLR.shape)
display(x_MLR.head())
# ## 4. Visualize the data using a scatter plot
import matplotlib.pyplot as plt

# Plot the first numeric predictor against house price
numeric_predictors = x_MLR.select_dtypes(include="number").columns

if len(numeric_predictors) > 0:
    plot_col = "area" if "area" in x_MLR.columns else
numeric_predictors[0]
    plt.figure(figsize=(8, 5))
    plt.scatter(x_MLR[plot_col], y_MLR, alpha=0.7)
    plt.xlabel(plot_col)
    plt.ylabel(target_col)
    plt.title(f"{plot_col} vs {target_col}")
    plt.grid(True)
    plt.show()
else:
    print("No numeric predictor is available for a scatter plot.")
# ## 5. Split the data into training and testing sets
from sklearn.model_selection import train_test_split

x_MLR_train, x_MLR_test, y_MLR_train, y_MLR_test = train_test_split(
    x_MLR, y_MLR, test_size=0.20, random_state=42
)

print("Training features:", x_MLR_train.shape)
print("Testing features:", x_MLR_test.shape)
print("Training target:", y_MLR_train.shape)
print("Testing target:", y_MLR_test.shape)
# ## 6. Train the Multiple Linear Regression model
from sklearn.linear_model import LinearRegression

MLR_model = LinearRegression()
MLR_model.fit(x_MLR_train, y_MLR_train)

```

```

print("Multiple Linear Regression model trained successfully.")
# ## 7. Display slopes (coefficients) and intercept
print("Intercept (b0):", MLR_model.intercept_)

coefficients_df = pd.DataFrame({
    "Feature": x_MLR.columns,
    "Slope / Coefficient": MLR_model.coef_
})

print("\nSlope values for all predictors:")
display(coefficients_df)
# ## 8. Make predictions and calculate accuracy metrics
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

y_MLR_pred = MLR_model.predict(x_MLR_test)

mae = mean_absolute_error(y_MLR_test, y_MLR_pred)
mse = mean_squared_error(y_MLR_test, y_MLR_pred)
rmse = mse ** 0.5
r2 = r2_score(y_MLR_test, y_MLR_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score:", r2)

metrics_df = pd.DataFrame({
    "Metric": ["MAE", "MSE", "RMSE", "R2 Score"],
    "Value": [mae, mse, rmse, r2]
})
display(metrics_df)
# ## 9. Visualize actual vs predicted house prices
import numpy as np
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
plt.scatter(y_MLR_test, y_MLR_pred, alpha=0.7)

min_val = min(y_MLR_test.min(), y_MLR_pred.min())
max_val = max(y_MLR_test.max(), y_MLR_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], "r--", label="Perfect
prediction")

plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Multiple Linear Regression: Actual vs Predicted")
plt.legend()
plt.grid(True)

```

```

plt.show()
# ## 10. Regression-line style visualization for one predictor
# For MLR, one single regression line cannot represent all predictors
# in one 2D plot.
# This visualization compares actual and model-predicted prices
# against one selected predictor.
if len(numeric_predictors) > 0:
    plot_col = "area" if "area" in x_MLR.columns else
numeric_predictors[0]

    temp = pd.DataFrame({
        plot_col: x_MLR_test[plot_col],
        "Actual Price": y_MLR_test,
        "Predicted Price": y_MLR_pred
    }).sort_values(plot_col)

    plt.figure(figsize=(9, 5))
    plt.scatter(temp[plot_col], temp["Actual Price"], alpha=0.6,
label="Actual data")
    plt.plot(temp[plot_col], temp["Predicted Price"], linewidth=2,
label="MLR predicted values")
    plt.xlabel(plot_col)
    plt.ylabel(target_col)
    plt.title(f"MLR Prediction Trend using {plot_col}")
    plt.legend()
    plt.grid(True)
    plt.show()
else:
    print("No numeric predictor is available for a scatter plot.")
# ## 11. Predict the price of a new house
# Example: create a new observation from an existing row.
# Replace the values below with your own house details when needed.

new_house = x_MLR.iloc[[0]].copy()

# Example changes only if these columns exist
if "area" in new_house.columns:
    new_house["area"] = 3000
if "bedrooms" in new_house.columns:
    new_house["bedrooms"] = 3
if "bathrooms" in new_house.columns:
    new_house["bathrooms"] = 2

predicted_price = MLR_model.predict(new_house)

print("Predicted house price:", predicted_price[0])
# ## Conclusion

# The experiment successfully trains a Multiple Linear Regression
# model for house-price prediction using `x_MLR` and `y_MLR`. It

```



```

includes basic preprocessing, visualization, train/test splitting,
model training, coefficients and intercept, accuracy metrics,
prediction visualization, and new-house prediction.
# # PART 2 – Polynomial Linear Regression

# This is a **separate pipeline** from the previous model.

# A degree-2 polynomial model uses:

# # Price = intercept + coefficient1 × Area + coefficient2 × Area2

# Required variable names:

# - `X_PLR`
# - `Y_PLR`
# ## 1. Import libraries and data preprocessing (basic)
# Imports for the separate Polynomial Linear Regression pipeline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# Use the dataset loaded at the beginning of the notebook.
# Basic preprocessing only: select area and price, then remove missing
rows.
data_PLR = df[['area', 'price']].copy()

print("Missing values before preprocessing:")
print(data_PLR.isnull().sum())

data_PLR = data_PLR.dropna()

print("\nMissing values after preprocessing:")
print(data_PLR.isnull().sum())

print("\nShape after preprocessing:", data_PLR.shape)
display(data_PLR.head())
# ## 2. Create `X_PLR` and `Y_PLR`
# X_PLR = input feature (area)
# Y_PLR = target (house price)

# Define data_PLR within this cell to resolve NameError

```

```

data_PLR = df[['area', 'price']].copy()
data_PLR = data_PLR.dropna()

X_PLR = data_PLR[['area']]
Y_PLR = data_PLR['price']

print("X_PLR shape:", X_PLR.shape)
print("Y_PLR shape:", Y_PLR.shape)
# ## 3. Visualize the original data with a scatter plot
plt.figure(figsize=(9, 6))
plt.scatter(X_PLR['area'], Y_PLR, alpha=0.6)
plt.xlabel('Area (sq. ft.)')
plt.ylabel('House Price')
plt.title('House Area vs House Price')
plt.grid(True)
plt.show()
# ## 4. Split the data into training and testing sets
X_train_PLR, X_test_PLR, Y_train_PLR, Y_test_PLR = train_test_split(
    X_PLR,
    Y_PLR,
    test_size=0.20,
    random_state=42
)

print("Training samples:", len(X_train_PLR))
print("Testing samples :", len(X_test_PLR))
# ## 5. Convert the input into polynomial features
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split # Added import

# Degree of polynomial
degree_PLR = 5

poly_PLR = PolynomialFeatures(
    degree=degree_PLR,
    include_bias=False
)

# Define X_train_PLR and X_test_PLR within this cell
X_train_PLR, X_test_PLR, Y_train_PLR, Y_test_PLR = train_test_split(
    X_PLR,
    Y_PLR,
    test_size=0.20,
    random_state=42
)

X_train_poly_PLR = poly_PLR.fit_transform(X_train_PLR)
X_test_poly_PLR = poly_PLR.transform(X_test_PLR)

```

```

print("Polynomial feature names:")
print(poly_PLR.get_feature_names_out(['area']))

print("\nTraining polynomial data shape:", X_train_poly_PLR.shape)
print("Testing polynomial data shape :", X_test_poly_PLR.shape)
# ## 6. Train the Polynomial Linear Regression model
model_PLR = LinearRegression()

model_PLR.fit(
    X_train_poly_PLR,
    Y_train_PLR
)

print("Polynomial Linear Regression model trained successfully.")
# ## 7. Get polynomial coefficients and intercept
coefficients_PLR = model_PLR.coef_
intercept_PLR = model_PLR.intercept_

print(f"Intercept: {intercept_PLR:.6f}")

for feature_name, coefficient in zip(
    poly_PLR.get_feature_names_out(['area']),
    coefficients_PLR
):
    print(f"Coefficient for {feature_name}: {coefficient:.6f}")

print("\nRegression equation:")
print(
    f"Price = ({coefficients_PLR[0]:.6f} × Area) "
    f"+ ({coefficients_PLR[1]:.6f} × Area2) "
    f"+ {intercept_PLR:.6f}"
)
# ## 8. Accuracy / evaluation metrics
Y_pred_PLR = model_PLR.predict(X_test_poly_PLR)

mae_PLR = mean_absolute_error(Y_test_PLR, Y_pred_PLR)
mse_PLR = mean_squared_error(Y_test_PLR, Y_pred_PLR)
rmse_PLR = np.sqrt(mse_PLR)
r2_PLR = r2_score(Y_test_PLR, Y_pred_PLR)

print("Polynomial Linear Regression Evaluation")
print("-----")
print(f"R2 Score : {r2_PLR:.6f}")
print(f"MAE      : {mae_PLR:,.2f}")
print(f"MSE      : {mse_PLR:,.2f}")
print(f"RMSE      : {rmse_PLR:,.2f}")
# ## 9. Visualize the polynomial regression curve
plt.figure(figsize=(9, 6))

# Actual observations

```

```

plt.scatter(
    X_PLR['area'],
    Y_PLR,
    alpha=0.5,
    label='Actual data'
)

# Smooth polynomial curve
X_curve_PLR = np.linspace(
    X_PLR['area'].min(),
    X_PLR['area'].max(),
    300
).reshape(-1, 1)

X_curve_poly_PLR = poly_PLR.transform(X_curve_PLR)
Y_curve_PLR = model_PLR.predict(X_curve_poly_PLR)

plt.plot(
    X_curve_PLR,
    Y_curve_PLR,
    linewidth=2,
    label=f'Polynomial curve (degree {degree_PLR})'
)

plt.xlabel('Area (sq. ft.)')
plt.ylabel('House Price')
plt.title('Polynomial Linear Regression: Area vs House Price')
plt.legend()
plt.grid(True)
plt.show()

# ## 10. Predict a house price using `X_PLR`
# Enter a new house area in square feet
new_area_PLR = 5000

# Create prediction input using the requested X_PLR naming convention
X_new_PLR = pd.DataFrame({'area': [new_area_PLR]})

# Convert the new input using the SAME polynomial transformer
X_new_poly_PLR = poly_PLR.transform(X_new_PLR)

predicted_price_PLR = model_PLR.predict(X_new_poly_PLR)[0]

print(f"Area: {new_area_PLR:,} sq. ft.")
print(f"Predicted house price: {predicted_price_PLR:,.2f}")
# ## Summary

# You now have two independent pipelines:

# | Pipeline | Input | Target | Model |
# |---|---|---|---|

```

```
# | Simple Linear Regression | `x_MLR` | `y_MLR` |
`LinearRegression`() |
# | Polynomial Linear Regression | `X_PLR` | `Y_PLR` |
`PolynomialFeatures` + `LinearRegression` |

# **Important:** The polynomial model is still called "polynomial
linear regression" because the model is linear in its learned
coefficients, even though the input contains polynomial terms.
# ---
```

<IPython.core.display.HTML object>

Saving Housing.csv to Housing.csv
Dataset shape: (545, 13)

```
{"summary":{"\n  \"name\": \"# ---\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"price\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 671416,\n        \"min\": 11410000,\n        \"max\": 13300000,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          12250000,\n          11410000,\n          13300000\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"area\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1158,\n        \"min\": 7420,\n        \"max\": 9960,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          8960,\n          7500,\n          7420\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"bedrooms\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 3,\n        \"max\": 4,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          3,\n          4\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"bathrooms\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1,\n        \"min\": 1,\n        \"max\": 4,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          2,\n          4\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"stories\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 2,\n        \"max\": 4,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          3,\n          4\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"mainroad\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"yes\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"guestroom\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"yes\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"
```

```

{"basement": {"properties": {"dtype": "category", "num_unique_values": 2, "samples": [{"yes": "yes"}], "semantic_type": "boolean", "description": "Is there a basement?"}, {"hotwaterheating": {"properties": {"dtype": "category", "num_unique_values": 1, "samples": [{"no": "no"}], "semantic_type": "boolean", "description": "Is there hot water heating?"}, {"airconditioning": {"properties": {"dtype": "category", "num_unique_values": 2, "samples": [{"no": "no"}], "semantic_type": "boolean", "description": "Is there air conditioning?"}, {"parking": {"properties": {"dtype": "number", "std": 0, "min": 2, "max": 3, "num_unique_values": 2, "samples": [{"3": 3}], "semantic_type": "number", "description": "Number of parking spaces"}, {"prefarea": {"properties": {"dtype": "category", "num_unique_values": 2, "samples": [{"no": "no"}], "semantic_type": "boolean", "description": "Is there a preferred area?"}, {"furnishingstatus": {"properties": {"dtype": "category", "num_unique_values": 2, "samples": [{"semi-furnished": "semi-furnished"}], "semantic_type": "boolean", "description": "Is the house semi-furnished?"}}], "type": "dataframe"}

```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 545 entries, 0 to 544
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	price	545 non-null	int64
1	area	545 non-null	int64
2	bedrooms	545 non-null	int64
3	bathrooms	545 non-null	int64
4	stories	545 non-null	int64
5	mainroad	545 non-null	object
6	guestroom	545 non-null	object
7	basement	545 non-null	object
8	hotwaterheating	545 non-null	object
9	airconditioning	545 non-null	object
10	parking	545 non-null	int64
11	prefarea	545 non-null	object
12	furnishingstatus	545 non-null	object

```
dtypes: int64(6), object(7)
```

```
memory usage: 55.5+ KB
```

```
Missing values before preprocessing:
```

```

price      0
area       0
bedrooms   0

```

```
bathrooms      0
stories         0
mainroad        0
guestroom       0
basement        0
hotwaterheating 0
airconditioning 0
parking         0
prefarea        0
furnishingstatus 0
dtype: int64
```

Missing values after preprocessing:

```
price          0
area           0
bedrooms       0
bathrooms      0
stories         0
mainroad        0
guestroom       0
basement        0
hotwaterheating 0
airconditioning 0
parking         0
prefarea        0
furnishingstatus 0
dtype: int64
```

Target column: price

x_MLR shape: (545, 13)

y_MLR shape: (545,)

```
{"summary": "{\n  \"name\": \"# ---\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"area\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1158,\n        \"min\": 7420,\n        \"max\": 9960,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          8960,\n          7500,\n          7420\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\": \"bedrooms\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0,\n          \"min\": 3,\n          \"max\": 4,\n          \"num_unique_values\": 2,\n          \"samples\": [\n            3,\n            4\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\": \"bathrooms\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 1,\n            \"min\": 1,\n            \"max\": 4,\n            \"num_unique_values\": 3,\n            \"samples\": [\n              2,\n              4\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\": \"stories\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 0,\n              \"min\": 2,\n
```

```

{"max": 4, "num_unique_values": 3, "samples": [3, 4], "semantic_type": "column", "description": "parking", "properties": {"dtype": "number", "std": 0, "min": 2, "max": 3, "num_unique_values": 2, "samples": [3, 2], "semantic_type": "column", "description": "mainroad_yes", "properties": {"dtype": "number", "std": 0.0, "min": 1.0, "max": 1.0, "num_unique_values": 1, "samples": [1.0], "semantic_type": "column", "description": "guestroom_yes", "properties": {"dtype": "number", "std": 0.447213595499958, "min": 0.0, "max": 1.0, "num_unique_values": 2, "samples": [1.0], "semantic_type": "column", "description": "basement_yes", "properties": {"dtype": "number", "std": 0.5477225575051662, "min": 0.0, "max": 1.0, "num_unique_values": 2, "samples": [1.0], "semantic_type": "column", "description": "hotwaterheating_yes", "properties": {"dtype": "number", "std": 0.0, "min": 0.0, "max": 0.0, "num_unique_values": 1, "samples": [0.0], "semantic_type": "column", "description": "airconditioning_yes", "properties": {"dtype": "number", "std": 0.4472135954999579, "min": 0.0, "max": 1.0, "num_unique_values": 2, "samples": [0.0], "semantic_type": "column", "description": "prefarea_yes", "properties": {"dtype": "number", "std": 0.5477225575051662, "min": 0.0, "max": 1.0, "num_unique_values": 2, "samples": [0.0], "semantic_type": "column", "description": "furnishingstatus_semi-furnished", "properties": {"dtype": "number", "std": 0.44721359549995804, "min": 0.0, "max": 1.0, "num_unique_values": 2, "samples": [1.0], "semantic_type": "column", "description": "furnishingstatus_unfurnished", "properties": {"dtype": "number", "std": 0.0, "min": 0.0, "max": 0.0, "num_unique_values": 1, "samples": [0.0], "semantic_type": "column", "description": " "}}], "type": "dataframe"}

```




Training features: (436, 13)
 Testing features: (109, 13)
 Training target: (436,)
 Testing target: (109,)
 Multiple Linear Regression model trained successfully.
 Intercept (b0): 260032.35760741215

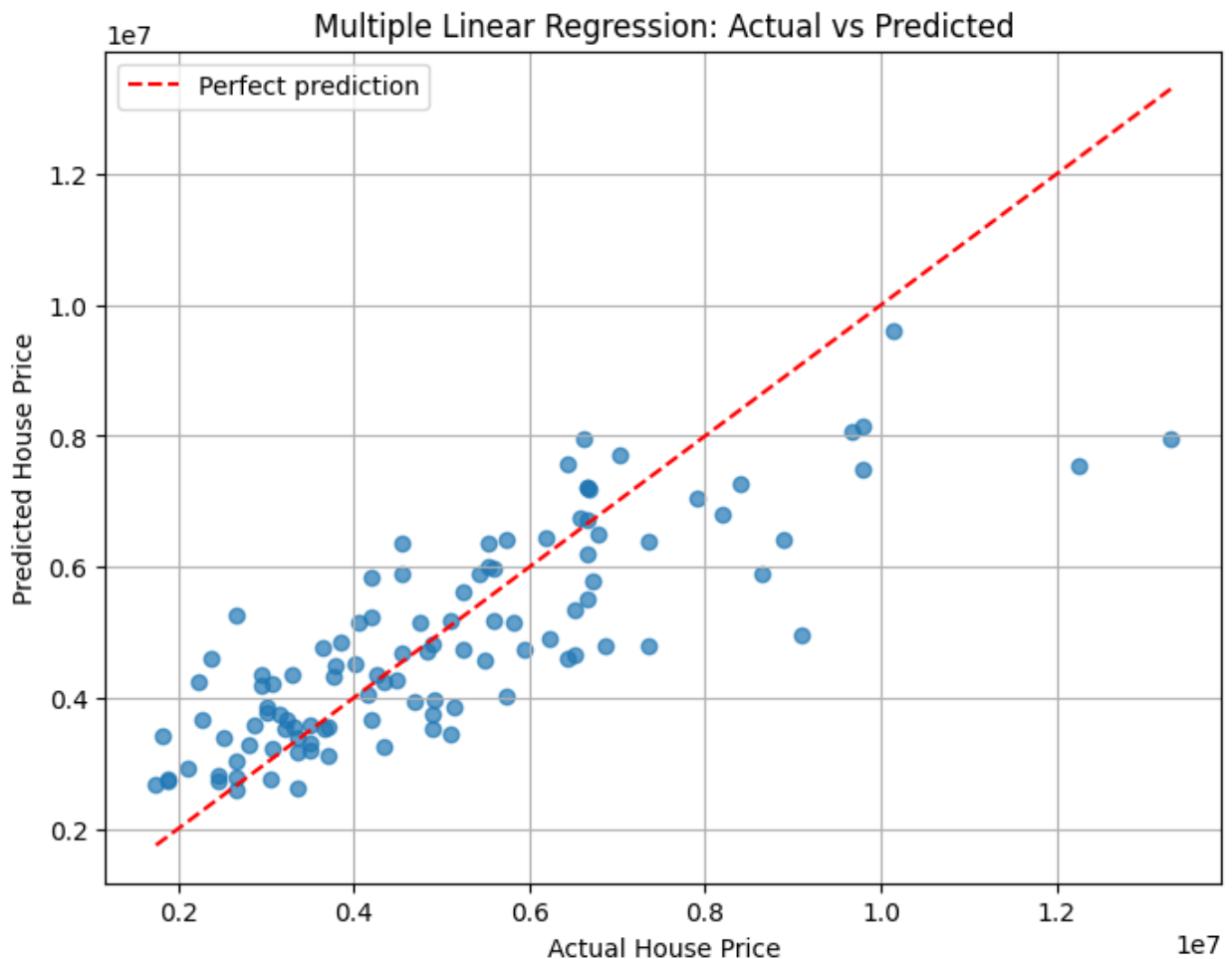
Slope values for all predictors:

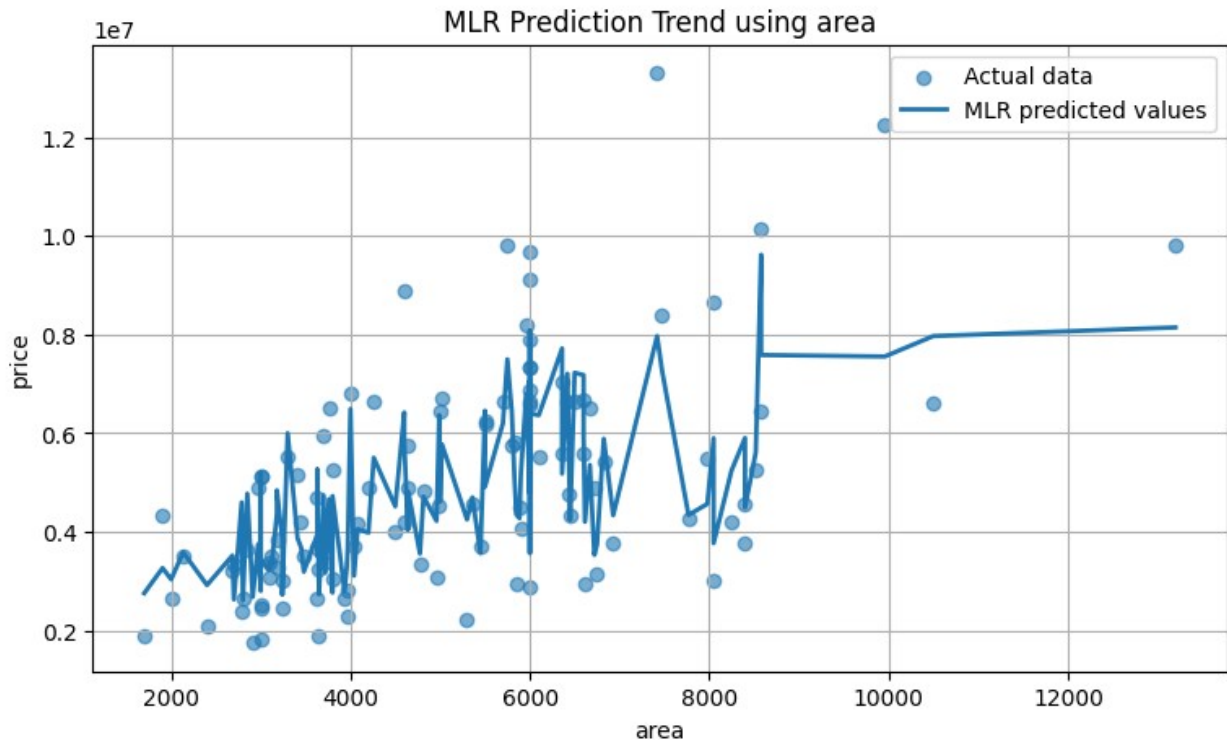
```

{"summary":{"\n  \"name\": \"coefficients_df\",\n  \"rows\": 13,\n  \"fields\": [\n    {\n      \"column\": \"Feature\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 13,\n        \"samples\": [\n          \"furnishingstatus_semi-furnished\",\n          \"airconditioning_yes\",\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        ]\n      },\n      \"column\": \"Slope / Coefficient\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 406105.3849875831,\n        \"min\": -413645.062228222,\n        \"max\": 1094444.7861005957,\n        \"num_unique_values\": 13,\n        \"samples\": [\n          -126881.81758510851,\n          791426.7358431868,\n          235.9688045413957\n        ]\n      },\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    }\n  ]\n}, \"type\": \"dataframe\", \"variable_name\": \"coefficients_df\"}
  
```

MAE : 970043.4039201637
MSE : 1754318687330.6643
RMSE : 1324506.9600914388
R² Score: 0.6529242642153184

```
{"summary":{"name": "metrics_df", "rows": 4, "fields": [{"column": "Metric", "properties": {"dtype": "string", "num_unique_values": 4, "samples": [{"R\u00b2 Score", "MSE", "MAE", "description": ""}], "semantic_type": ""}, {"column": "Value", "properties": {"dtype": "number", "std": 877158961240.3413, "min": 0.6529242642153184, "max": 1754318687330.6643, "num_unique_values": 4, "samples": [1754318687330.6643, 0.6529242642153184, 970043.4039201637]}, {"column": "semantic_type": "", "description": ""}], "type": "dataframe", "variable_name": "metrics_df"}
```





Predicted house price: 6848515.308680032

Missing values before preprocessing:

```
area      0
price     0
dtype: int64
```

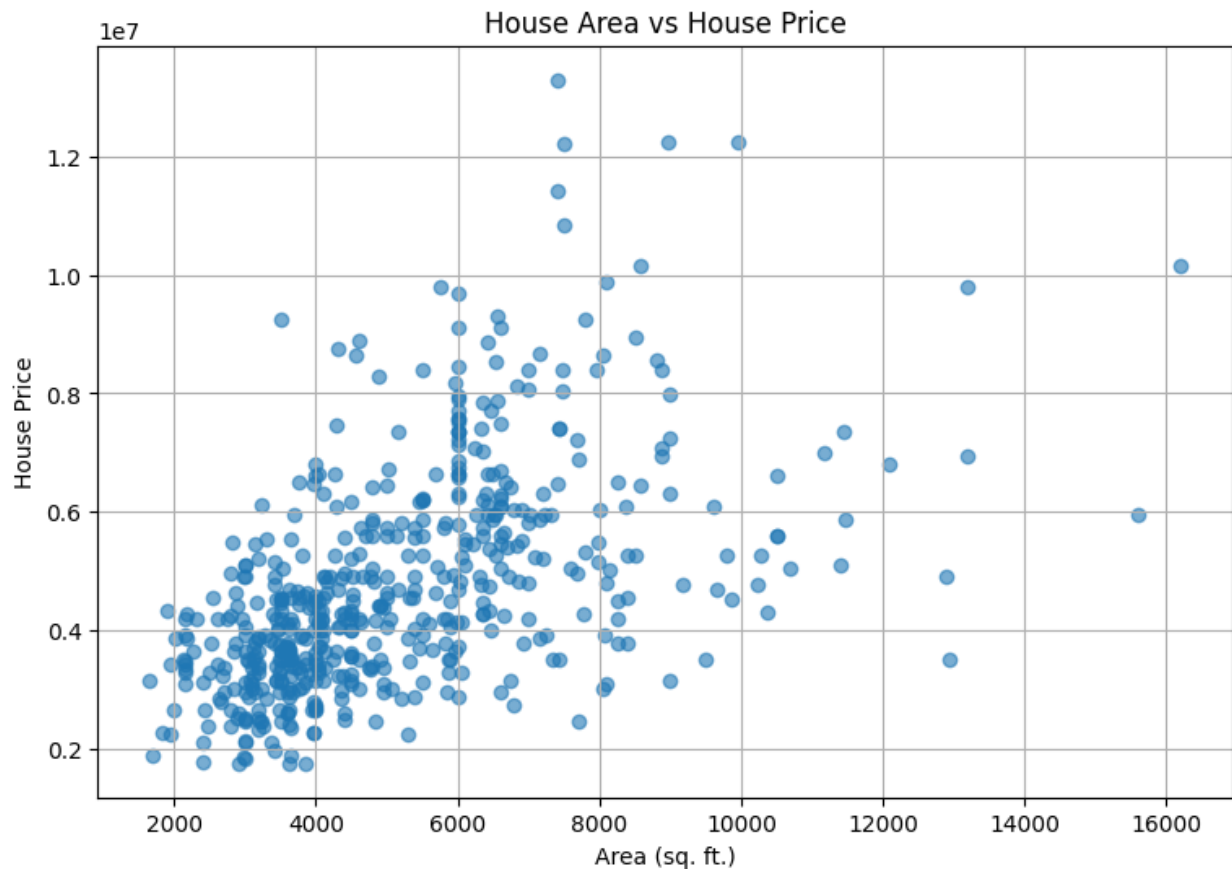
Missing values after preprocessing:

```
area      0
price     0
dtype: int64
```

Shape after preprocessing: (545, 2)

```
{"summary":{"\n  \"name\": \"# ---\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"area\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1158,\n        \"min\": 7420,\n        \"max\": 9960,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          8960,\n          7500,\n          7420\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"price\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 671416,\n        \"min\": 11410000,\n        \"max\": 13300000,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          12250000,\n          11410000,\n          13300000\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}, \"type\": \"dataframe\"}
```

```
X_PLR shape: (545, 1)
Y_PLR shape: (545,)
```



```
Training samples: 436
Testing samples : 109
Polynomial feature names:
['area' 'area^2' 'area^3' 'area^4' 'area^5']
```

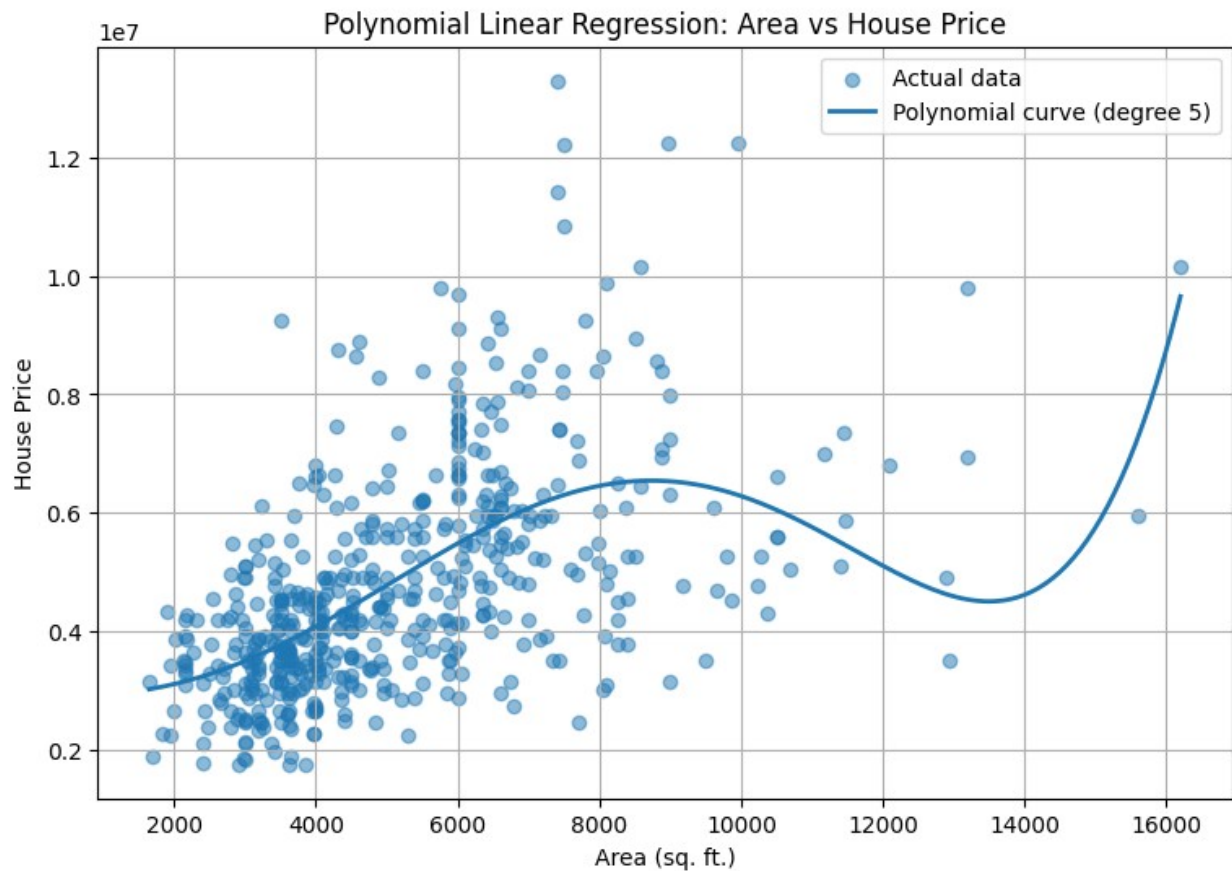
```
Training polynomial data shape: (436, 5)
Testing polynomial data shape : (109, 5)
Polynomial Linear Regression model trained successfully.
Intercept: 2891229.172335
Coefficient for area: 0.000000
Coefficient for area^2: 0.000000
Coefficient for area^3: 0.000036
Coefficient for area^4: -0.000000
Coefficient for area^5: 0.000000
```

```
Regression equation:
Price = (0.000000 × Area) + (0.000000 × Area2) + 2891229.172335
Polynomial Linear Regression Evaluation
```

```
-----
```

R² Score : 0.257706
MAE : 1,465,969.36
MSE : 3,751,977,584,851.90
RMSE : 1,937,002.22

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/  
validation.py:2739: UserWarning: X does not have valid feature names,  
but PolynomialFeatures was fitted with feature names  
warnings.warn(
```



Area: 5,000 sq. ft.
Predicted house price: 4,771,363.53
Error: Runtime no longer has a reference to this dataframe, please re-run this cell and try again.